

Kernel Engineering Agent Framework

- 핵심 기여점(Contribution): Architect가 '시스템 모델(System Model)'을 바탕으로 가설을 세우고, Executor가 이를 증명하는 구조를 통한 nano-vllm의 cuTile 구현 및 성능 평가

Overview

GPU 커널 엔지니어링, 컴파일러 최적화, 분산 시스템 튜닝과 같이 "실측 데이터가 정적 설계보다 더 중요해지는 영역"을 위한 2계층(2-Tier) 에이전트 프레임워크입니다.

기존의 Task Decomposition 방식(예: TileGym)을 넘어, **Scientist(가설 수립) - Laboratory(실험 및 측정)** 구조를 도입하여 GPU 커널 개발을 통제된 과학 실험 과정으로 모델링합니다. 본 프로젝트는 이 프레임워크를 통해 cuTile 기반 커널 프로그래밍의 효율성과 성능 우수성을 체계적으로 검증합니다.

Architecture: 2-Tier System

Role	Environment	Core Responsibilities
Architect (Scientist)	Host machine (No GPU), Project root	도메인 특화 지식(Skill) 축적 및 시스템 모델(System Model) 유지 관리.
소스코드 역설계 및 작업 목표에 따른 가설 (<code>HYPOTHESIS.md</code>)과 실험 계획 (<code>PLAN.md</code>) 생성.		
실험 결과를 분석하여 시스템 모델 및 장기 기억 업데이트.		
Executor (Laboratory)	Target machine (GPU RTX 4060), Sub-folder	1차 멘탈 리허설 및 Tool Calling을 통한 소스코드 편집.
GPU 환경에서 코드 컴파일, 실행, 프로파일링(Nsight Compute) 수행.		
실제 측정된 메트릭을 바탕으로 실측 보고서(<code>TRACE.md</code> , <code>RESULT.md</code>) 생성.		

Core Workflow: Cybernetic Feedback Loop

GPU 최적화에서 가장 어려운 점은 설계의 타당성을 코드를 읽는 것만으로는 알 수 없다는 것입니다. 따라서 본 프레임워크는 제어공학의 폐루프 시스템(Closed Loop System)과 유사한 **증거 기반(Evidence-Oriented)** 피드백 워크플로우를 따릅니다. GPU에서 도출된 결과를 "진실의 원천(Source of Truth)"으로 삼습니다.

- Hypothesis (Architect):** `DESIGN.md`, `RULES.md`를 바탕으로 기대 성능(예: Shared Memory Tile 적용 시 대역폭 15% 향상)을 명시한 실험 스펙 생성.
- Experiment & Observation (Executor):** 타겟 GPU 환경에서 코드를 수정하고 벤치마크를 실행하여 커널 시간, 점유율(Occupancy), 메모리 대역폭 등의 실측 트레이스 캡처.
- Refinement (Architect):** 예상 지표와 실측 지표를 비교 분석하여 새로운 가설 수립 및 다음 실험 루프 지시.

Evaluation Roadmap

이 프레임워크의 유효성과 cuTile의 성능 우위를 입증하기 위해, 다음 3단계로 난이도를 높이며 실험을 진행합니다. 이 과정에서 획득한 데이터는 최종적으로 Kernel Engineering 방법론에 관한 논문의 실증 데이터로 활용됩니다.

Phase 1: FMHA (Flash Multi-Head Attention)

- **목표:** 기존 Attention 구현을 cuTile 기반으로 변환하고 베이스라인 성능 확보.
- **검증 포인트:** 4x4 타일 크기 조정 및 스레드 블록 할당(Thread block assignment) 최적화를 통한 메모리 접근 패턴 개선 확인.
- **평가 지표:** Kernel Execution Time, Shared Memory 활성화도, DRAM 트래픽 감소율.

Phase 2: LLM Forward Path (From Scratch)

- **목표:** 기초적인 언어 모델의 Forward Path 전체를 cuTile을 활용하여 밑바닥부터 구현.
- **검증 포인트:** 행렬 곱셈(MatMul) 최적화 및 연산 그래프 내에서의 타일링 전략 유효성 검증.
- **평가 지표:** End-to-end Forward Pass Time, TFLOPS, Resource Utilization.

Phase 3: nano-vLLM Integration

- **목표:** 실질적인 추론 서버 환경을 모사한 nano-vLLM 구조에 cuTile 기반 커널 통합.
- **검증 포인트:** PagedAttention과 같은 고급 메모리 관리 기법 및 KV 캐싱 최적화 시 cuTile의 적용 가능성과 성능 향상 입증.
- **평가 지표:** Token Generation Latency, Throughput (Tokens/sec), KV Cache 메모리 단편화 감소율.

제안 구조

GPU 커널 엔지니어링, 컴파일러 최적화, 분산 시스템 튜닝처럼 "실측 데이터가 설계보다 더 중요해지는 영역"에서는 TileGym식 Task Decomposition보다 이 Scientist-Laboratory 구조가 훨씬 자연스럽고 확장성도 높다.

DESIGN.md + SKILL.md + Context Graph 는 사실상 **Scientist의 이론 모델(Knowledge State)** 혹은 시스템 모델이고, GPU 머신에서 생성되는 Trace/Report는 **실험 데이터(Evidence State)** 이다.

2계층 Kernel Engineering Agent

[Architect 혹은 Scientist 계층]

- Domain Specific Skill 축적
- System Model (invariants + functional description)
- 장기 기억 (design documents, insights from the experiments and trace log)
- 실행 환경: Host machine w/o any GPU, project root folder
- Role:
 - 주어진 소스코드 기준으로 역설계 진행
 - 작업 목표에 따른 plan.md 생성
 - Executor의 trace log 분석 결과를 바탕으로 skill.md 개선

[Executor 혹은 Laboratory 계층]

- plan.md (Workflow) 파싱 및 1차 mental 리허설
- Tool Calling (소스코드 편집)
- 실행 환경: Target machine w/ GPU RTX4060, sub folder에서 agent 실행
- Role
 - plan.md 실행 및 trace log capture
 - trace log 분석
 - 실험 보고서 생성 혹은 반복 작업

2계층 Workflow

Architect



```
Plan
↓
Executor Loop
↓
Trace
↓
Architect
```

이는 **Cybernetic Feedback Workflow** 로서,
Architect는 '가설 생성'을 담당한다. Executor는 실제 GPU 환경에서

```
Compile
Run
Profile
Measure
```

실행한다.

그리고

```
Trace
Metrics
Failure
```

를 반환한다.

Architect는

```
예상
vs
실측
```

을 비교한다.

이 구조는 사실 제어공학(Control Theory)의

```
Closed Loop System
```

과 유사하다.

```
Hypothesis
↓
Experiment
↓
Observation
↓
Refinement
```

GPU 개발은 사실상 과학 실험

GPU 최적화에서 가장 어려운 점은
설계가 맞는지
코드를 읽어서는 알 수 없다는 것이다.
그래서 GPU 개발은
일반 소프트웨어 개발보다
과학 실험에 가깝다.

즉

Hypothesis
Experiment
Observation

루프가 핵심이다.

그래서 Architect가 가져야 할 것

나는 보다 더 중요한 것이 있다고 본다.

Architect는 plan.md 혹은 Hypothesis.md 를 생성한다.

Experiment: E-023

Goal:
Reduce DRAM traffic

Hypothesis:
Shared Memory Tile

Expected:
15~20% speedup

Metrics:
Kernel Time
Occupancy
Bandwidth

Success:
>10% improvement

이게 있어야

Executor 결과를 해석할 수 있다.

Executor는 코더가 아니라 실험실

많은 사람들이 Executor를

코드 생성기

로 생각한다.

하지만 GPU Kernel 개발에서는

Experiment Operator

가 더 적절하다.

Executor 책임

Compile

Run

Nsight Compute

Collect Trace

Generate Report

코드 작성보다 측정 품질이 중요하다.

내가 추천하는 구조

현재 구상에서 조금 확장하면

Architect (Host)

DESIGN.md

RULES.md

HYPOTHESIS.md

↓

Experiment Spec

↓

Executor (GPU Machine)

Code Modify

Build

Run

Profile

Measure

↓

TRACE.md

RESULT.md

↓

Architect

Analysis

New Hypothesis

Evidence-Oriented

GPU를 "진실의 원천"으로 둔다.

Architect

↓

GPU Reality



Executor

이다.

Scientist (Architect)

가설 생성



Laboratory (Executor)

실험 수행



Measurements



Scientist

모델 수정

참조

- TileGym 문서는 기본적으로 '문제를 어떻게 분해할 것인가'에 집중한다.

Problem



Analyzer



Sub-Agent 들



Composer

형태의 **Task Decomposition Workflow** <https://github.com/NVIDIA/TileGym/blob/main/skills/tilegym-cutile-python/orchestration/workflow.md>